Experiment No. 0

Aim :

To implement NumPy, Pandas, Matplotlib & Seaborn for Machine Learning.

Theory :

1. Dataset Source

Dataset Name: **Students performance in exams**

Source Platform: Kaggle

Dataset Link :
https://www.kaggle.com/code/julianowakowska/students-performance-in-exams/input

2. Dataset Description

The dataset consists of student performance records with both categorical and numerical attributes.

Features:

- gender – Student gender
- race/ethnicity – Student category
- parental level of education – Parent education level
- lunch – Lunch type
- test preparation course – Course completion status
- math score – Numerical exam score
- reading score – Numerical exam score
- writing score – Numerical exam score

Target Variable (Used in ML Part):

- math score (predicted using reading and writing scores)

Dataset Characteristics:

- Contains categorical and numerical data
- Suitable for statistical analysis and visualization
- Used for demonstrating preprocessing and machine learning workflow

3. Mathematical Formulation of the Algorithm

A simple Linear Regression model was used.

Linear Regression Equation:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \epsilon$$

Evaluation Metric:

$$R^2 = 1 - \frac{\sum(y - \hat{y})^2}{\sum(y - \bar{y})^2}$$

R^2 score measures how well the model explains variance in the data.

4. Algorithm Limitations

- Linear Regression assumes a linear relationship between variables.
- Sensitive to outliers in the dataset.
- Cannot capture complex nonlinear patterns.
- Performance decreases when features are highly correlated or insufficient.

5. Methodology / Workflow

- Downloaded the Students Performance dataset from Kaggle.
- Uploaded the CSV file into Google Colab.
- Loaded dataset using Pandas library.
- Performed exploratory data analysis using Pandas functions such as info() and describe().
- Used NumPy to create an average score feature from exam marks.
- Visualized data using Matplotlib:
    - Histogram
    - Scatter plots
- Generated advanced visualizations using Seaborn:
    - Heatmap
- Selected features and applied Linear Regression model.
- Predicted test data values and evaluated performance using R^2 score.
- Applied GridSearchCV for hyperparameter tuning.

6. Performance Analysis

The model performance was evaluated using the R^2 score, which indicates how well the predicted values match the actual values.

Observations:

- Reading and writing scores showed strong correlation with math score.
- Visualizations such as heatmap and scatter plots helped identify feature relationships.
- The Linear Regression model achieved a moderate R^2 score, showing reasonable prediction capability.
- Seaborn visualizations improved understanding of data distribution and outliers.

7. Hyperparameter Tuning

Hyperparameter tuning was performed using GridSearchCV with Ridge Regression as an extension of Linear Regression.

Process:

- Multiple alpha values were tested.
- Cross-validation was used to find the best parameter.
- The optimal alpha improved model stability and reduced overfitting.

Impact:

- Slight improvement in R^2 score
- Better generalization on unseen data

Output :

[1] 2s

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

[4] 0s

```
df = pd.read_csv("StudentsPerformance.csv")
df.head()
```

	gender	race/ethnicity	parental level of education	lunch	test preparation course	math score	reading score	writing score
0	female	group B	bachelor's degree	standard	none	72	72	74
1	female	group C	some college	standard	completed	69	90	88
2	female	group B	master's degree	standard	none	90	95	93
3	male	group A	associate's degree	free/reduced	none	47	57	44
4	male	group C	some college	standard	none	76	78	75

Next steps: Generate code with df New interactive sheet

[8]

```python
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 8 columns):
 #   Column                       Non-Null Count  Dtype
---  ------                       --------------  -----
 0   gender                       1000 non-null   object
 1   race/ethnicity               1000 non-null   object
 2   parental level of education  1000 non-null   object
 3   lunch                        1000 non-null   object
 4   test preparation course      1000 non-null   object
 5   math score                   1000 non-null   int64
 6   reading score                1000 non-null   int64
 7   writing score                1000 non-null   int64
dtypes: int64(3), object(5)
memory usage: 62.6+ KB
```

[9]

```python
df.describe()
```

	math score	reading score	writing score
count	1000.00000	1000.000000	1000.000000
mean	66.08900	69.169000	68.054000
std	15.16308	14.600192	15.195657
min	0.00000	17.000000	10.000000
25%	57.00000	59.000000	57.750000
50%	66.00000	70.000000	69.000000
75%	77.00000	79.000000	79.000000
max	100.00000	100.000000	100.000000

[10]

```python
df.shape
```

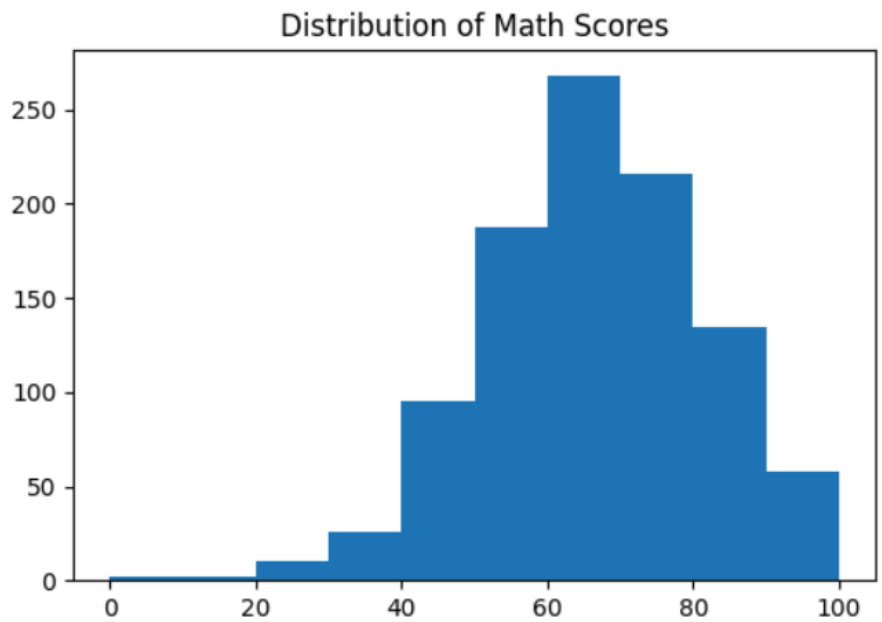
```
(1000, 8)
```

[11]

```python
df["average_score"] = np.mean(
    df[["math score","reading score","writing score"]], axis=1
)
```

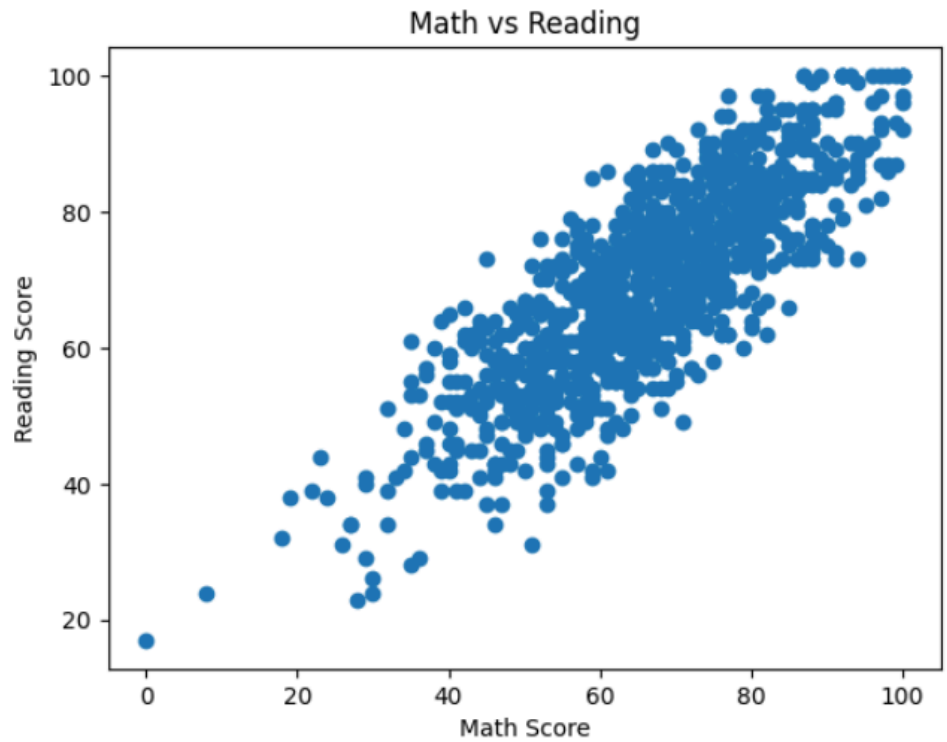
[12]
0s

```
plt.figure(figsize=(6,4))
plt.hist(df["math score"])
plt.title("Distribution of Math Scores")
plt.show()
```



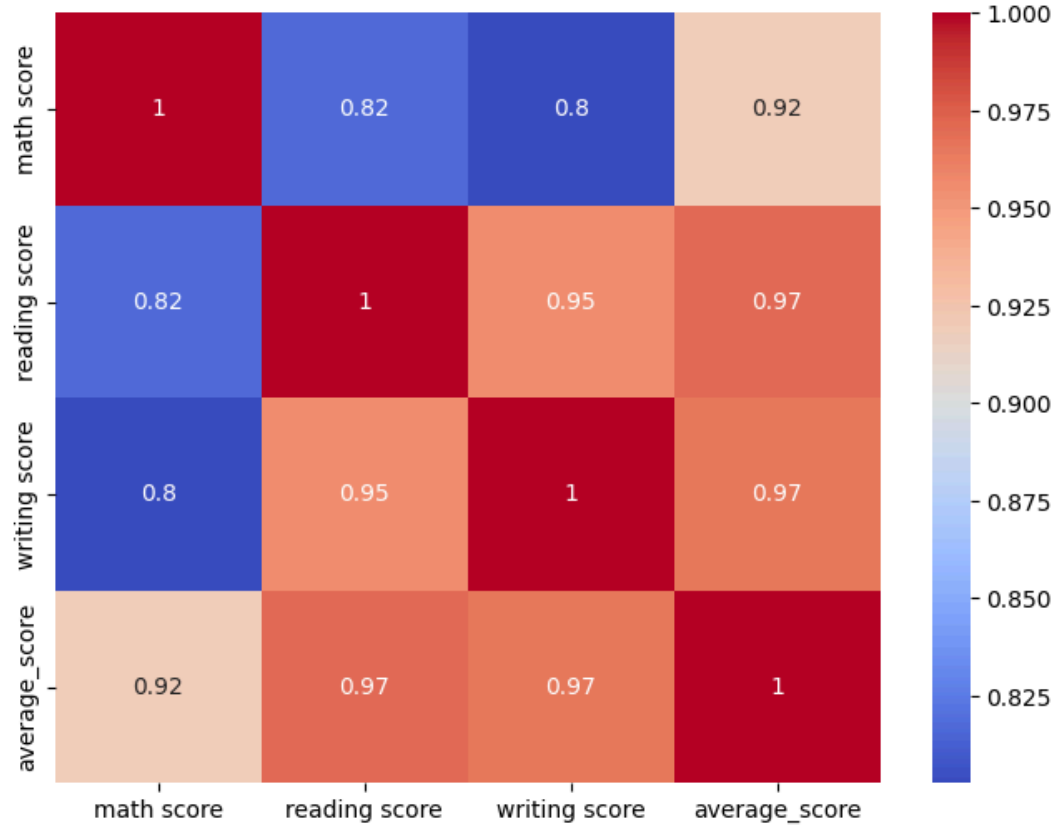
[13]
✓ 0s

```python
plt.scatter(df["math score"], df["reading score"])
plt.xlabel("Math Score")
plt.ylabel("Reading Score")
plt.title("Math vs Reading")
plt.show()
```



[14]

```python
plt.figure(figsize=(8,6))
sns.heatmap(df.select_dtypes(include='number').corr(), annot=True, cmap="coolwarm")
plt.show()
```



[17]

```python
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
```

[18]

```python
X = df[["reading score","writing score"]]
y = df["math score"]
```

[19]

```python
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2,random_state=42)
```

[20]

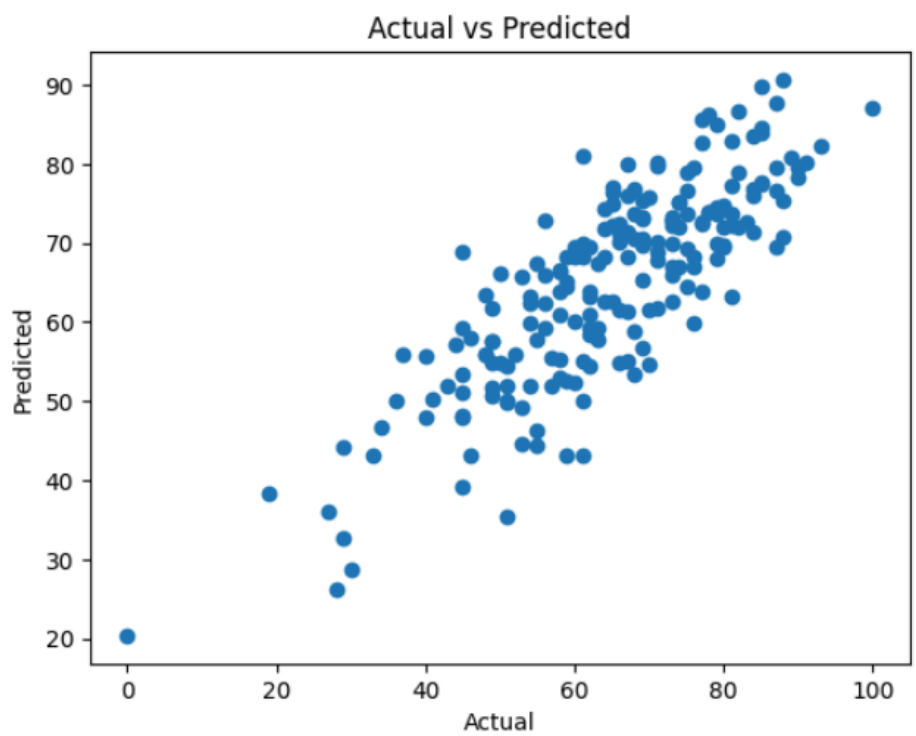
```python
model = LinearRegression()
model.fit(X_train,y_train)
y_pred = model.predict(X_test)

print("R2:", r2_score(y_test,y_pred))
```

```
R2: 0.6825697127424626
```

[21]
0s

```python
plt.scatter(y_test, y_pred)
plt.xlabel("Actual")
plt.ylabel("Predicted")
plt.title("Actual vs Predicted")
plt.show()
```



[22]
0s

```python
from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import Ridge

params = {"alpha":[0.1,1,10]}

grid = GridSearchCV(Ridge(), params, cv=5)
grid.fit(X_train,y_train)

print(grid.best_params_)
```

```
{'alpha': 10}
```

Conclusion :

The experiment demonstrated how NumPy, Pandas, Matplotlib, and Seaborn can be effectively used for data preprocessing, visualization, and machine learning tasks. Pandas and NumPy enabled efficient data handling, while Matplotlib and Seaborn provided meaningful visual insights. The Linear Regression model showed the relationship between exam scores and allowed prediction of student performance. Overall, the experiment highlighted the importance of visualization and preprocessing in building effective machine learning models.